

External Project Report on Computer Organization and Architecture (EET 2211)

Design A 16-bit Calculator (Arithmetic,
Logical And Number System Conversion)
Using 8086 Assembly Language



Submitted by

AMULYA SHRIVASTAVA	Reg. No.:	2341019456
ANURAG PRADHAN	Reg. No.:	2341016359
AMAAN AHMED	Reg. No.:	2341001055
ISHAN ARYADIT	Reg. No.:	2341001110

B. Tech. CSE 4th Semester (Section – 23412F3)

INSTITUTE OF TECHNICAL EDUCATION AND RESEARCH
(FACULTY OF ENGINEERING)

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY), BHUBANESWAR,
ODISHA

Declaration

We, The Undersigned Students Of B. Tech. Of CSE Department Hereby Declare That We Own The Full Responsibility For The Information, Results Etc. Provided In This Project Titled “Design An 16-Bit Calculator (Arithmetic, Logical And Number System Conversion) Using 8086 Assembly Language” Submitted To Siksha ‘O’ Anusandhan Deemed To Be University, Bhubaneswar For The Partial Fulfillment Of The Subject Computer Organization And Architecture (EET 2211). We Have Taken Care In All Respect To Honor The Intellectual Property Right And Have Acknowledged The Contribution Of Others For Using Them In Academic Purpose And Further Declare That In Case Of Any Violation Of Intellectual Property Right Or Copyright We, As The Candidate(s), Will Be Fully Responsible For The Same.

(AMULYA SHRIVASTAVA)

Registration No.:

2341019456

(ANURAG PRADHAN)

Registration No.:

2341016359

(AMAAN AHMED)

Registration No.:

2341001055

(ISHAN ARYADIT)

Registration No.:

2341001110

DATE: 20/05/2025

PLACE: I.T.E.R, SOA (BHUBANESWAR)

Abstract

This Project Presents The Design And Implementation Of An 16-Bit Calculator Using 8086 Assembly Language, Capable Of Performing Arithmetic Operations, Logical Functions, And Number System Conversions. The Calculator Supports Basic Arithmetic Operations Including Addition, Subtraction, Multiplication, Division, And Remainder Calculation, While Also Incorporating Logical Operations Such As AND, OR, XOR, And NOT. In Addition To These Functions, The Calculator Effectively Converts Numerical Results Into Binary, Decimal And Hexadecimal Number Systems For Output Display. Developed Specifically For The Intel 8086 Microprocessor Architecture, The Program Utilizes Direct Register Manipulation And DOS Interrupts To Interact With The User Through Keyboard Input And Screen Output. The Implementation Ensures Robust Error Handling, Including Division By Zero Detection, To Maintain Program Stability. This Project Provides A Practical Understanding Of Low-Level Programming Concepts, Bitwise Operations, And Real-Time User Interaction In Assembly Language. It Acts As An Educational Tool For Those Interested In Exploring Computer Architecture, Embedded Systems, And The Inner Workings Of Computational Logic At The Hardware Level.

Contents

Serial No.	Chapter No.	Title of the Chapter	Page No.
1.	1	Introduction	01
2.	2	Problem Statement	02
3.	3	Methodology	03
4.	4	Implementation	04-11
5.	5	Results And Interpretation	12-14
6.	6	Conclusion	15
7.		References	16

1. Introduction

The 8086 Microprocessor, Developed By Intel, Is A Powerful 16-Bit Processor That Laid The Foundation For X86 Architecture. Programming In 8086 Assembly Language Provides Valuable Insight Into Hardware-Level Computation, Precise Register Manipulation, And Direct Memory Access. This Project Focuses On Designing An 16-Bit Calculator Using 8086 Assembly Language, Supporting Arithmetic Operations Such As Addition, Subtraction, Multiplication, Division, And Modulus; Logical Operations Including AND, OR, XOR, And NOT; Along With Number System Conversions To Binary, Decimal, And Hexadecimal Formats. The Calculator Accepts Two 8-Bit Hexadecimal Inputs From The Keyboard And Uses DOS Interrupts For Input And Output Handling, Enabling Real-Time User Interaction. Each Operation Is Performed Using Low-Level Instructions With Proper Register Usage, Error Handling For Cases Like Division By Zero, And Bitwise Manipulation For Logical Results. Conversion Routines Are Carefully Implemented To Accurately Display Binary And Decimal Equivalents Of The Result. Through This Implementation, The Project Demonstrates The Efficiency And Transparency Of Low-Level Programming In Performing Fundamental Computational Tasks, Making It An Excellent Educational Tool For Understanding Microprocessor Architecture, Instruction-Level Execution, And Assembly Language Programming.

2. Problem Statement

The Objective Is To Design An 16-Bit Calculator Using 8086 Assembly Language That Performs Arithmetic Operations Such As Addition, Subtraction, Multiplication, Division, And Remainder Calculation, Along With Logical Operations Including AND, OR, XOR, And NOT. The Calculator Also Supports Number System Conversions, Displaying The Result In Binary, Decimal And Hexadecimal Formats. It Accepts Two 8-Bit Hexadecimal Digits As Input Through The Keyboard, Processes Them Using 8086 Instructions, And Displays The Output Using DOS Interrupts. Division By Zero Is Appropriately Handled With A Custom Error Message. This Project Demonstrates Practical Implementation Of Low-Level Programming Concepts, Including Register Operations, Bitwise Manipulation, And Direct Interaction With System Interrupts. It Enhances Understanding Of Microprocessor Functionality, Efficient Use Of CPU Registers, And Real-Time Execution In A DOS-Based Environment.

3. Methodology

Define Requirements: Identify And Categorize Required Operations— Arithmetic, Logical, And Number System Conversions.

Setup Environment: Use An Emulator Or Assembler Like MASM/TASM To Write And Test 8086 Assembly Code.

Input Handling: Capture 16-Bit User Input From The Keyboard Using DOS Interrupts (INT 21h).

Data Validation: Ensure Inputs Are Valid 16-Bit Values And Fall Within Acceptable Number System Formats.

Arithmetic Operations: Implement Addition, Subtraction, Multiplication, And Division Using 8086 Instructions.

Logical Operations: Program Bitwise Operations Such As AND, OR, XOR, And NOT.

Conversion Functions: Develop Routines To Convert Numbers Between Binary, Decimal And Hexadecimal.

Result Display: Use INT 21h Functions To Display Results Clearly On The Screen.

User Interface Loop: Create A Looped Menu System For Continuous User Interaction And Operation Selection.

Continuation Loop: Create A Looped Menu System Which Makes The User To Input Choices To Exit/Halt The Program Or Re-Run The Loop.

Testing And Debugging: Test Each Function Separately, Debug Errors, And Ensure Accurate Computation And Display.

4. Implementation

Here Is A Basic **Implementation** Of An **16-bit Calculator** In **8086 Assembly** Using **EMU8086**. It Includes A Simple **Menu System**, And Performs **Arithmetic Operations**, **Logical Operations**, And **Number System Conversions** For 16-bit Inputs.

The Assembly Code Is As Follows:

```
.model small
.stack 100h

.data
    msg1      db 13, 10, 'Enter First Hex Digit (0-9, A-F): $'
    msg2      db 13, 10, 'Enter Second Hex Digit (0-9, A-F): $'
    msgAdd     db 13, 10, 10, 'Addition: $'
    msgSub     db 13, 10, 'Subtraction: $'
    msgMul     db 13, 10, 'Multiplication: $'
    msgDiv     db 13, 10, 'Division (Quotient): $'
    msgRem     db 13, 10, 'Division (Remainder): $'
    msgAnd     db 13, 10, 'AND: $'
    msgOr      db 13, 10, 'OR: $'
    msgXor     db 13, 10, 'XOR: $'
    msgNot1    db 13, 10, 'NOT Of First: $'
    msgNot2    db 13, 10, 'NOT Of Second: $'
    msgBin1    db 13, 10, 10, 'First In Binary: $'
    msgBin2    db 13, 10, 'Second In Binary: $'
    msgDec1    db 13, 10, 'First In Decimal: $'
    msgDec2    db 13, 10, 'Second In Decimal: $'
    newline    db 13, 10, '$'
    hexChars   db '0123456789ABCDEF'
    divByZero  db 13, 10, 'Division By Zero!', 13, 10, '$'
    promptRepeat db 13, 10, 10, 'Enter 0 To Exit Or 1 To Repeat: $'

.code
main:
    mov ax, @data
    mov ds, ax

program_loop:
    ; Print Message 1 And Read First Hex Digit
    lea dx, msg1
    mov ah, 09h
    int 21h
    call ReadHex
```

```

mov bl, al    ; Store First Number In BL

; Print Message 2 And Read Second Hex Digit
lea dx, msg2
mov ah, 09h
int 21h
call ReadHex
mov bh, al    ; Store Second Number In BH

; ----- ARITHMETIC OPERATIONS -----
; Addition
lea dx, msgAdd
mov ah, 09h
int 21h
mov al, bl
add al, bh
call PrintHex

; Subtraction
lea dx, msgSub
mov ah, 09h
int 21h
mov al, bl
sub al, bh
call PrintHex

; Multiplication
lea dx, msgMul
mov ah, 09h
int 21h
mov al, bl
mov ah, 0     ; Clear AH For Multiplication
mul bh       ; AX = AL * BH
call PrintHex ; Prints AL (Lower Byte Of Result)

; Division (With Check For Division By Zero)
lea dx, msgDiv
mov ah, 09h
int 21h
cmp bh, 0
je divisionError

mov al, bl
mov ah, 0
div bh       ; AL = Quotient, AH = Remainder

```

```
mov cl, ah    ; Store Remainder
```

```
; Print Quotient  
call PrintHex
```

```
; Print Remainder  
lea dx, msgRem  
mov ah, 09h  
int 21h  
mov al, cl  
call PrintHex
```

```
jmp afterDivision ; Skip Error Label
```

```
divisionError:  
    lea dx, divByZero  
    mov ah, 09h  
    int 21h
```

```
afterDivision:
```

```
; ----- LOGICAL OPERATIONS -----
```

```
; AND  
lea dx, msgAnd  
mov ah, 09h  
int 21h  
mov al, bl  
and al, bh  
call PrintHex
```

```
; OR  
lea dx, msgOr  
mov ah, 09h  
int 21h  
mov al, bl  
or al, bh  
call PrintHex
```

```
; XOR  
lea dx, msgXor  
mov ah, 09h  
int 21h  
mov al, bl  
xor al, bh  
call PrintHex
```

```
; NOT (First Number)
lea dx, msgNot1
mov ah, 09h
int 21h
mov al, bl
not al
and al, 0FFh ; Mask To 8 Bits
call PrintHex
```

```
; NOT (Second Number)
lea dx, msgNot2
mov ah, 09h
int 21h
mov al, bh
not al
and al, 0FFh ; Mask to 8 bits
call PrintHex
```

```
; ----- NUMBER SYSTEM CONVERSIONS -----
```

```
; First Number In Binary
```

```
lea dx, msgBin1
mov ah, 09h
int 21h
mov al, bl
call PrintBinary
```

```
; Second Number In Binary
```

```
lea dx, msgBin2
mov ah, 09h
int 21h
mov al, bh
call PrintBinary
```

```
; First Number In Decimal
```

```
lea dx, msgDec1
mov ah, 09h
int 21h
mov al, bl
call PrintDecimal
```

```
; Second Number In Decimal
```

```
lea dx, msgDec2
mov ah, 09h
int 21h
```

```

mov al, bh
call PrintDecimal

; Ask User If They Want To Repeat
lea dx, promptRepeat
mov ah, 09h
int 21h

; Read User Choice
mov ah, 01h
int 21h

cmp al, '1'
je program_loop ; Repeat If Input Is '1'

; Exit Program If Input Is Not '1'
mov ah, 4Ch
int 21h

; Subroutine: ReadHex
; Reads One Hex Digit (0-9, A-F) And Converts To Number (0-15)
; Returns: AL = Numeric Value (0-15)
ReadHex:
    mov ah, 01h
    int 21h      ; Input Character In AL

    ; Convert ASCII To Hex Value
    cmp al, '0'
    jb InvalidChar
    cmp al, '9'
    jbe Digit
    cmp al, 'A'
    jb InvalidChar
    cmp al, 'F'
    jbe AlphaUpper
    cmp al, 'a'
    jb InvalidChar
    cmp al, 'f'
    jbe AlphaLower

InvalidChar:
    mov al, 0      ; Default To 0 On Invalid Input
    ret

Digit:

```

```
sub al, '0' ; Convert '0'-'9' To 0-9
ret
```

AlphaUpper:

```
sub al, 'A' ; Convert 'A'-'F' To 10-15
add al, 10
ret
```

AlphaLower:

```
sub al, 'a' ; Convert 'a'-'f' To 10-15
add al, 10
ret
```

; Subroutine: PrintHex

; Prints AL (16-Bit Value) In 2-HEX

PrintHex:

```
push ax
push bx
push cx
push dx
```

```
mov cl, al ; Save Original Value
lea bx, hexChars ; Point To Hex Character Table
```

; High Nibble

```
mov al, cl
shr al, 4
and al, 0Fh ; Ensure Only 4 Bits
xlat ; Convert To Hex Character
mov dl, al
mov ah, 02h
int 21h
```

; Low Nibble

```
mov al, cl
and al, 0Fh
xlat ; Convert To Hex Character
mov dl, al
mov ah, 02h
int 21h
```

```
pop dx
pop cx
pop bx
pop ax
```

```
ret
```

```
; Subroutine: PrintBinary
```

```
; Prints AL (8-bit Value) In Binary Format
```

```
PrintBinary:
```

```
push ax
```

```
push bx
```

```
push cx
```

```
push dx
```

```
mov bl, al    ; Save Original Value
```

```
mov cx, 8     ; 8 Bits To Print
```

```
PrintBit:
```

```
rol bl, 1     ; Rotate Left Through Carry
```

```
mov dl, '0'
```

```
jnc PrintZero
```

```
mov dl, '1'
```

```
PrintZero:
```

```
push ax
```

```
mov ah, 02h
```

```
int 21h
```

```
pop ax
```

```
loop PrintBit
```

```
pop dx
```

```
pop cx
```

```
pop bx
```

```
pop ax
```

```
ret
```

```
; Subroutine: PrintDecimal
```

```
; Prints AL 16-bit Value In Decimal Format
```

```
PrintDecimal:
```

```
push ax
```

```
push bx
```

```
push cx
```

```
push dx
```

```
mov bl, al    ; Save Original Value
```

```
mov ah, 0     ; Clear AH For Division
```

```
mov al, bl    ; Restore Value To AX (AH=0, AL=value)
```

```
; Handle 3 Digits (0-255)
```

```
mov cl, 100
```

```
div cl      ; AL = Hundreds Digit, AH = Remainder
mov ch, ah  ; Save Remainder
add al, '0' ; Convert To ASCII
mov dl, al
mov ah, 02h
int 21h     ; Print Hundreds Digit
```

```
mov al, ch  ; Restore Remainder
mov ah, 0
mov cl, 10
div cl      ; AL = Tens Digit, AH = Units Digit
mov bx, ax ; Save Both Digits
```

```
add bl, '0' ; Convert Tens To ASCII
mov dl, bl
mov ah, 02h
int 21h     ; Print Tens Digit
```

```
add bh, '0' ; Convert Units To ASCII
mov dl, bh
mov ah, 02h
int 21h     ; Print Units Digit
```

```
pop dx
pop cx
pop bx
pop ax
ret
```

```
end main
```

5. Results & Interpretation

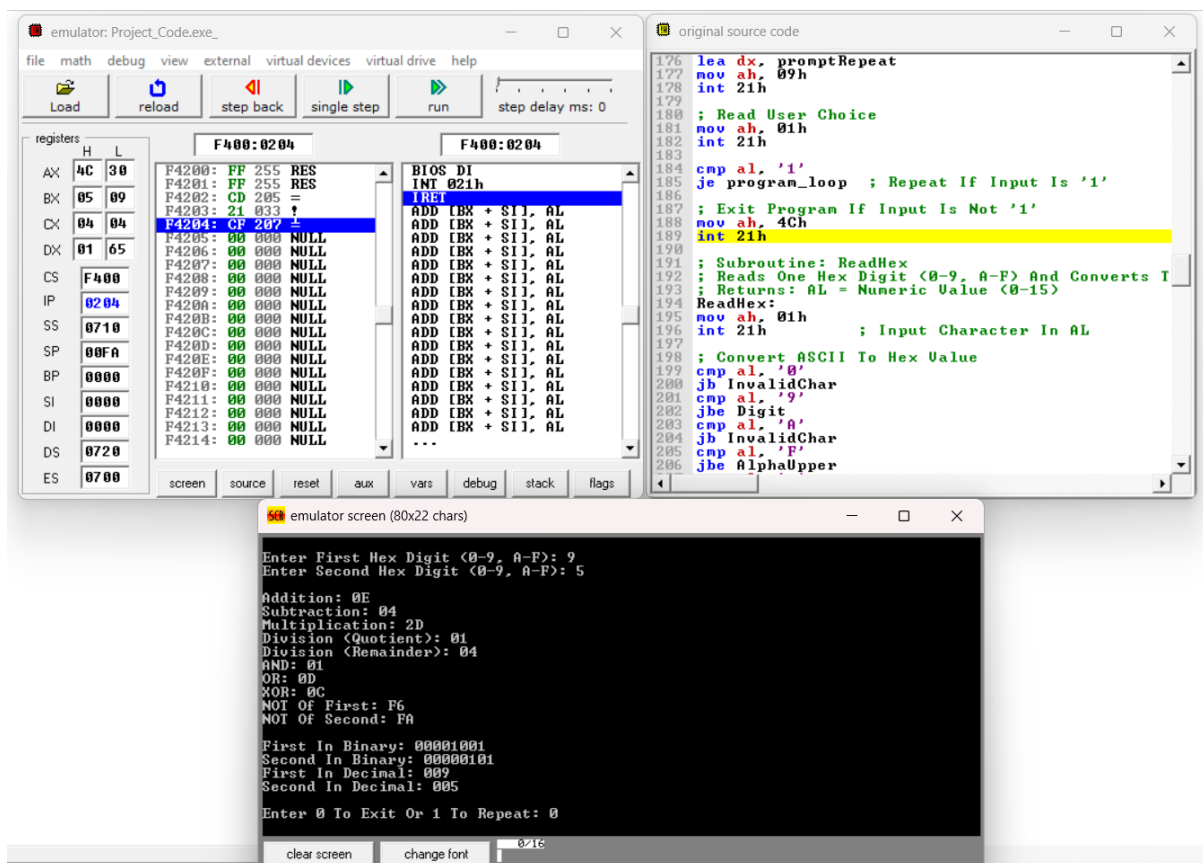
Result -1 :

Let's Assume You Enter The Following Characters When Prompted:

- First Digit: 9 (In Hexadecimal)
- Second Digit: 5 (In Hexadecimal)
- Choice To Exit Or Continue: 0

Output :

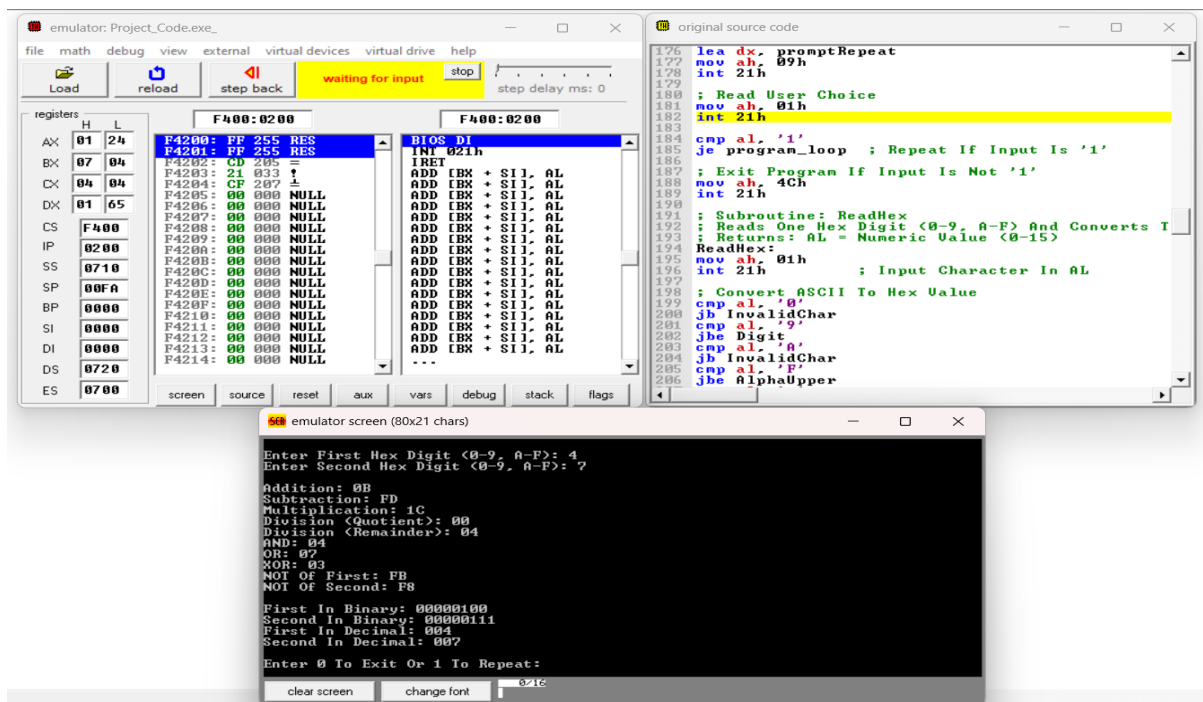
Result – 2 :



Let's Assume You Enter The Following Characters When Prompted:

- First Digit: 4 (In Hexadecimal)
- Second Digit: 7 (In Hexadecimal)
- Choice To Exit Or Continue: 1

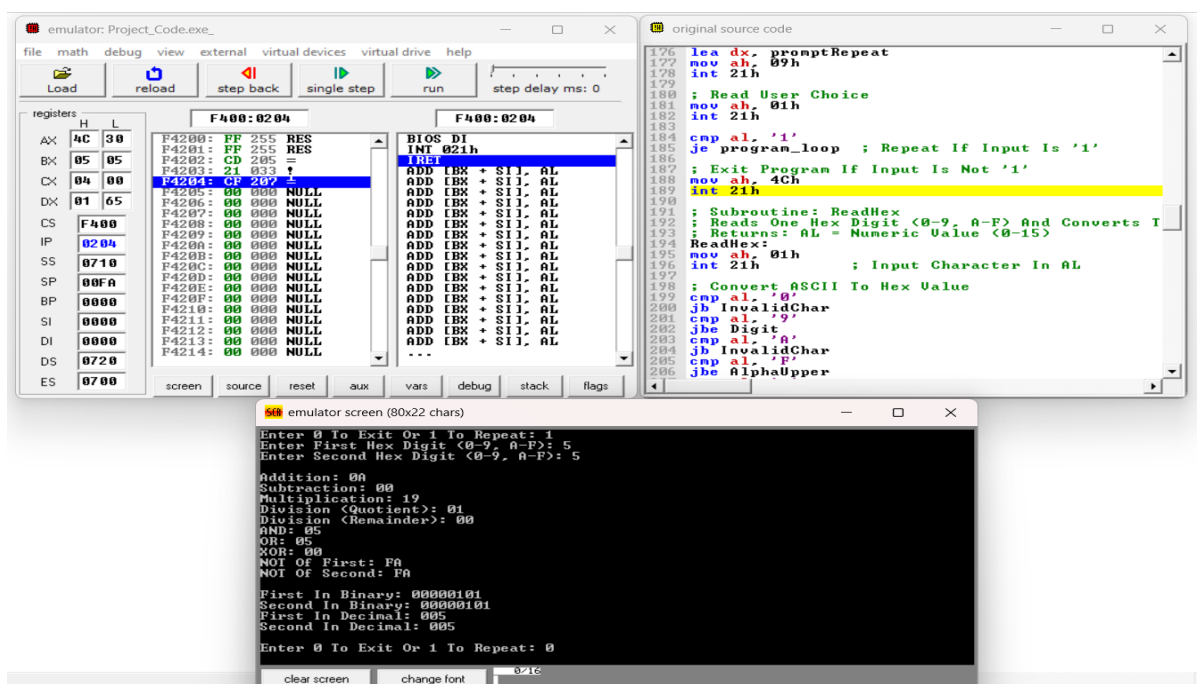
Output :



Let's Assume You Enter The Following Characters When Prompted:

- First Digit: 5 (In Hexadecimal)
- Second Digit: 5 (In Hexadecimal)
- Choice To Exit Or Continue: 0

Output :

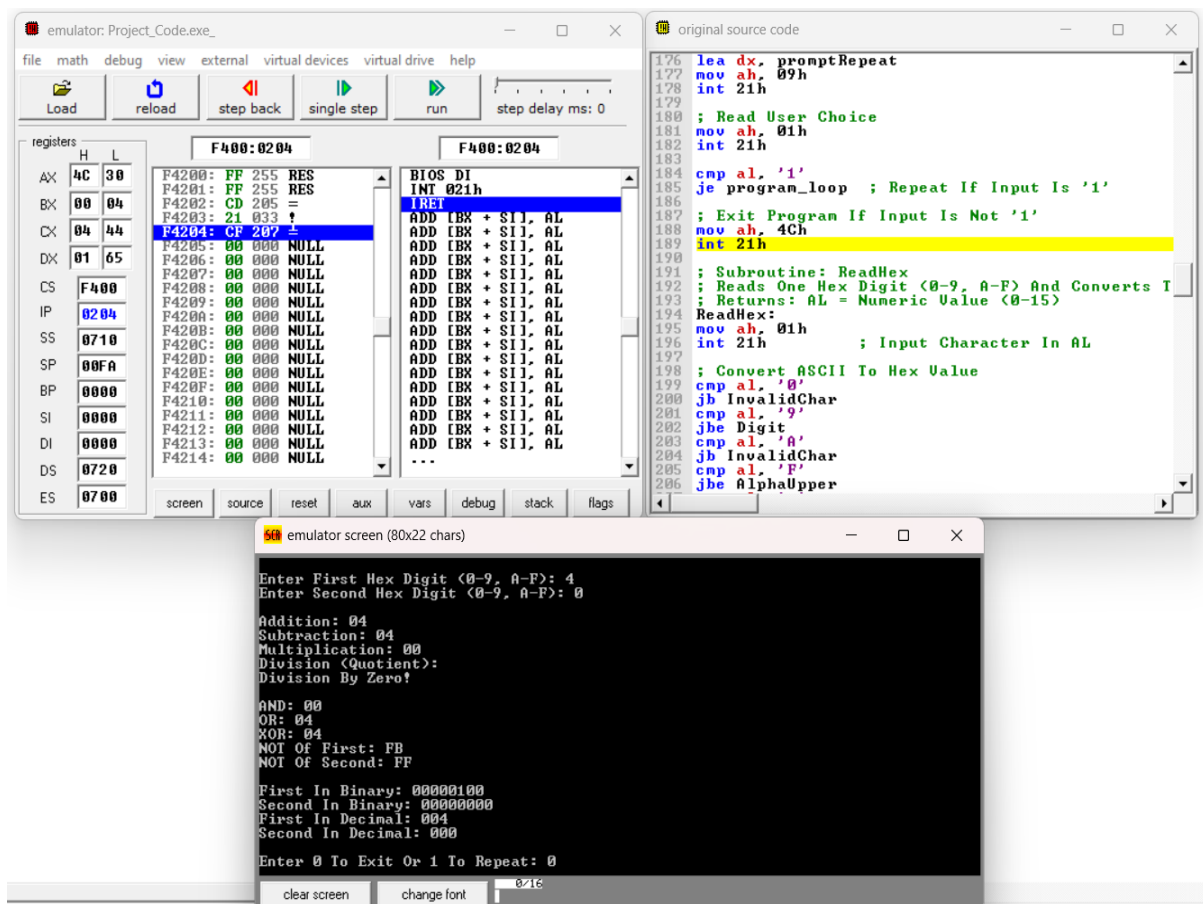


Result – 3 :

Let's Assume You Enter The Following Characters When Prompted:

- First Digit: 4 (In Hexadecimal)
- Second Digit: 0 (In Hexadecimal)
- Choice To Exit Or Continue: 0

Output :



6. Conclusion

The Design And Implementation Of An 16-Bit Calculator Using 8086 Assembly Language Effectively Demonstrate The Capabilities And Limitations Of Low-Level Programming For Arithmetic, Logical Operations, And Number System Conversions. This Project Highlights How Fundamental Computational Tasks Such As Addition, Subtraction, Bitwise Operations (AND, OR, XOR, NOT), And Conversions Between Binary, Decimal, And Hexadecimal Can Be Directly Controlled And Executed By The Processor. By Using Assembly Language, The Program Ensures Efficient Use Of CPU Registers And Memory, Giving Insight Into How Hardware-Level Operations Are Performed. The Project Also Reinforces The Understanding Of Interrupt-Driven Input/Output Through DOS Services, Allowing Interaction With Users Via Keyboard And Screen.

Despite Its Simplicity, This Calculator Serves As An Educational Tool To Appreciate The Complexity Behind High-Level Calculator Applications And Emphasizes The Importance Of Data Representation In Computer Systems. Handling Number System Conversions Within 8086 Assembly Deepens Comprehension Of Binary Arithmetic And Encoding, Which Is Critical For Embedded Systems And Hardware Design. Although Modern Systems Rely On Higher-Level Languages For Such Tasks, Mastering These Assembly Concepts Builds A Strong Foundation In Computer Architecture And System Programming. Overall, The Project Successfully Integrates Arithmetic, Logical, And Conversion Functions Into A Compact, Interactive Program, Showcasing The Practical Use Of 8086 Assembly In Real-World Applications.

References

(As Per The IEEE Recommendations)

- Intel Corporation, Intel® 64 And IA-32 Architectures Software Developer's Manual, Vol. 1, 2019.
- R. Hyde, The Art Of Assembly Language, 2nd Ed. San Francisco, CA, USA: No Starch Press, 2010.
- M. Beck, "8086 Assembly Language Programming Tutorial," 2021. Available.
- J. S. Irvine, Assembly Language For Intel-Based Computers, 6th Ed. Boston, MA, USA: Pearson, 2012.